

Calculus with Python

Prof Happy AI, 2026

Nội dung chính của file tập trung vào việc tự động hóa các bài tập giải đạo hàm thông qua thư viện **SymPy** và **NumPy**. Dưới đây là giải thích chi tiết các thành phần:

+1

1. Khởi tạo và thiết lập môi trường

Tài liệu bắt đầu bằng việc nhập các thư viện cần thiết để tính toán toán học và vẽ đồ thị:

+1

- **SymPy**: Dùng để tính toán các biểu thức toán học dưới dạng ký hiệu (symbolic).
- **NumPy**: Dùng để xử lý mảng dữ liệu số.
- **Matplotlib**: Dùng để trực quan hóa hàm số và đạo hàm.

2. Các bài tập thực hành (Exercises)

• Bài tập 1: Tạo hàm số ngẫu nhiên (Generate a random function)

- Sử dụng mã nguồn để tạo ra một đa thức có bậc ngẫu nhiên từ 1 đến 5.
- Kết hợp ngẫu nhiên với các hàm siêu việt (transcendental functions) như $\cos(x)$, $\sin(x)$, $\log(x)$, và $\exp(x)$.
- Mục tiêu là tạo ra một biểu thức toán học bất kỳ để người học thực hành tính đạo hàm. Ví dụ trong file tạo ra hàm: $-x^3 + 5x^2 + 5x + e^x + \cos(x) - 5$.

+1

+1

• Bài tập 2: Chuyển đổi và Vẽ đồ thị (Lambdify and plot)

- **sym.lambdify**: Chuyển đổi biểu thức toán học từ dạng ký hiệu sang dạng hàm số mà Python có thể tính toán bằng số thực.
- **Tính đạo hàm thực nghiệm**: Sử dụng `np.diff` để tính toán sự thay đổi giữa các điểm dữ liệu nhằm mô phỏng đạo hàm trên đồ thị.
- **Trực quan hóa**: Vẽ hai biểu đồ chồng lên nhau: biểu đồ phía trên là hàm số gốc $f(x)$ và biểu đồ phía dưới là đạo hàm của nó (df).

+2

• Bài tập 3: Tính đạo hàm bằng SymPy (Derivative in SymPy)

- Sử dụng lệnh `sym.diff(fx)` để máy tính tự động giải và đưa ra kết quả đạo hàm chính xác của hàm số đã tạo.
- Trong ví dụ cuối của file, kết quả đạo hàm được in ra là $\sin(x) + \cos(x)$.

3. Mục đích giáo dục

File này giúp người học không bao giờ thiếu bài tập thực hành bằng cách sử dụng Python để tạo ra "vô số bài tập đạo hàm". Thay vì giải các bài tập cố định trong sách giáo khoa, người học có thể sử dụng code này để:

1. Tự tạo đề bài ngẫu nhiên.
2. Kiểm tra kết quả tính toán của mình bằng máy tính.
3. Hiểu mối quan hệ hình học giữa hàm số và đạo hàm thông qua hình ảnh đồ thị.

✓ COURSE: Master calculus 1 using Python: derivatives and applications

SECTION: Differentiation fundamentals

LECTURE: CodeChallenge: Infinite derivatives exercises

TEACHER: Mike X Cohen, sincxpress.com

COURSE URL: udemy.com/course/pycalc1_x/?couponCode=202307

Start coding or [generate](#) with AI.

```
import numpy as np
import sympy as sym
import matplotlib.pyplot as plt

# better image resolution
import matplotlib_inline.backend_inline
matplotlib_inline.backend_inline.set_matplotlib_formats('svg')
```

Start coding or [generate](#) with AI.

✓ Exercise 1: Generate a random function

```
# create variable x
x = sym.symbols('x')

# random polynomial order
polyOrder = np.random.randint(1,5)

# list of other functions to include
funList = [ sym.cos(x),sym.sin(x),sym.log(x),sym.exp(x) ]

# initialize the function
fx = 0

# add the polynomial terms
for i in range(polyOrder):
    fx += np.random.randint(-5,6)*x**i

# add the transcendental functions
for f in np.random.choice(funList,2,replace=False):
    fx += np.random.choice((-1,1))*f
```

```
print(f'Differentiate this!\n')
fx
```

Differentiate this!

$$-x^3 + 5x^2 + 5x + e^x + \cos(x) - 5$$

Start coding or [generate](#) with AI.

✓ Exercise 2: Lambdify and plot

```
# "default" domain
D = [-3,3]

# possibly change start of domain
domain = sym.calculus.util.continuous_domain(fx,x,sym.S.Reals)
if domain.start==0:
    D[0] = .001
```

```
# convert expression to function
fxfun = sym.lambdify(x,fx)

# compute function values and empirical derivative
xx = np.linspace(D[0],D[1],1234)
```

```

y = fxfun(xx)
dy = np.diff(y) / np.mean(np.diff(xx))

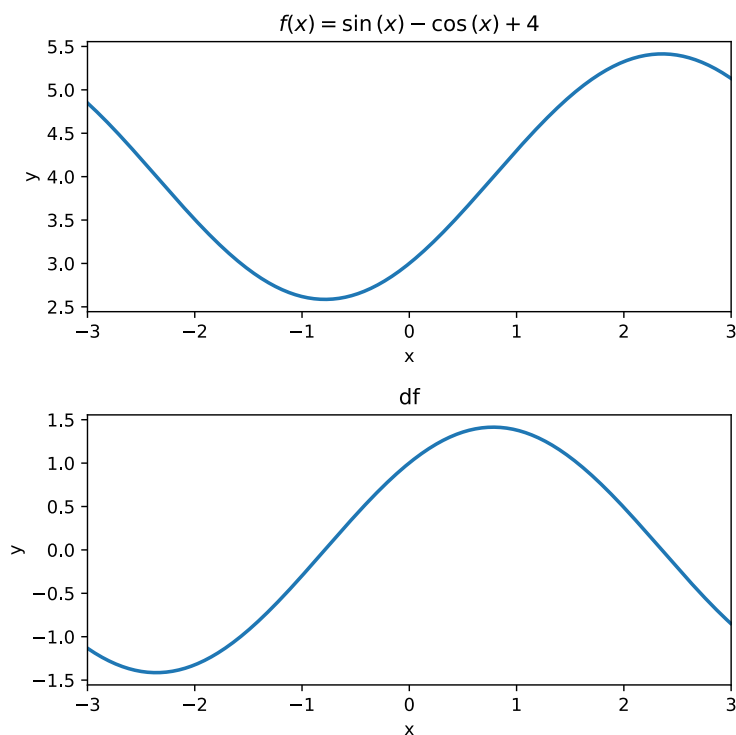
# plot the function
_,axs = plt.subplots(2,1,figsize=(6,6))
axs[0].plot(xx,y,linewidth=2)
axs[0].set_title(f'$f(x)={\text{sym.latex}(fx)}$')

# and the derivative
axs[1].plot(xx[:-1],dy,linewidth=2)
axs[1].set_title('df')

# prettify both axes
for a in axs:
    a.set_xlim(xx[[0,-1]])
    a.set_xlabel('x')
    a.set_ylabel('y')

plt.tight_layout()
plt.show()

```



Start coding or [generate](#) with AI.

Exercise 3: Derivative in sympy

```

print(f"Here's the derivative!\n")
sym.diff(fx)

```

Here's the derivative!

$\sin(x) + \cos(x)$

Start coding or [generate](#) with AI.

Tệp tài liệu bạn vừa tải lên là nội dung từ một bài giảng lập trình Python (dạng Google Colab) tập trung vào việc **tính đạo hàm của các hàm lượng giác** bằng phương pháp số học (numerical) và ký hiệu (symbolic).

+1

Dưới đây là phần giải thích chi tiết các nội dung chính trong tệp:

1. Thông tin chung về bài học

- **Chủ đề:** Master calculus 1 using Python (Làm chủ Giải tích 1 với Python).
- **Nội dung cụ thể:** Đạo hàm của các hàm lượng giác (Trig derivatives).
- **Công cụ sử dụng:** `numpy` (tính toán số), `sympy` (toán học ký hiệu), và `matplotlib` (vẽ đồ thị).

2. Các bài tập thực hành (Exercises)

Bài tập 1 & 2: Tính đạo hàm bằng phương pháp số (Numerical Differentiation)

Trong phần này, mã nguồn thực hiện tính đạo hàm của hàm $\cos(x)$ và $\sin(x)$ bằng cách sử dụng hiệu số chia cho khoảng cách nhỏ $dx = 0.01$.

+1

- **Cách làm:** Sử dụng hàm `np.diff(fx) / dx` để ước tính đạo hàm.

+1

- **Kết quả đồ thị:**
 - Đạo hàm của $\cos(x)$ khớp với đồ thị của hàm $-\sin(x)$.
 - Đạo hàm của $\sin(x)$ khớp với đồ thị của hàm $\cos(x)$.

Bài tập 3: Tính tuần hoàn của đạo hàm lượng giác (Cyclicity)

Bài tập này sử dụng thư viện `sympy` để chứng minh rằng đạo hàm của các hàm lượng giác lặp lại sau mỗi 4 lần thực hiện.

+1

- **Chu kỳ:**

1. $\cos(x)' = -\sin(x)$

$$2. (-\sin(x))' = -\cos(x)$$

$$3. (-\cos(x))' = \sin(x)$$

$$4. \sin(x)' = \cos(x) \text{ (quay lại hàm ban đầu)}$$

Bài tập 4: Đạo hàm của hàm lượng giác ngược (Inverse trig derivatives)

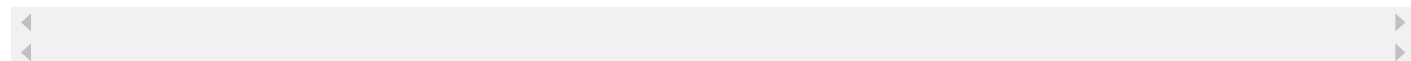
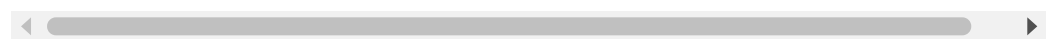
Phần này tính đạo hàm cho hàm $\arccos(x)$ (arc cosine).

+2

- Đạo hàm bậc nhất của $\arccos(x)$ là: $-\frac{1}{\sqrt{1-x^2}}$.
- Tài liệu cũng liệt kê các đạo hàm bậc cao hơn (đến bậc 4), cho thấy các hàm này trở nên phức tạp dần và **không có tính tuần hoàn** đơn giản như các hàm lượng giác thông thường.

3. Tóm tắt các hàm quan trọng trong Code

Thư viện	Hàm/Lệnh	Mục đích
Numpy	<code>np.diff()</code>	Tính sự chênh lệch giữa các phần tử để ước tính đạo hàm.
Sympy	<code>sym.diff()</code>	Tính đạo hàm chính xác theo công thức toán học (ký hiệu).
Matplotlib	<code>plt.plot()</code>	Vẽ đồ thị hàm số và các đạo hàm để so sánh trực quan.



Tóm lại, tệp này là một hướng dẫn thực hành giúp người học hiểu sâu về bản chất đạo hàm lượng giác thông qua việc lập trình và trực quan hóa bằng đồ thị.

✓ COURSE: Master calculus 1 using Python: derivatives and applications

SECTION: Differentiation fundamentals

LECTURE: CodeChallenge: trig derivatives

TEACHER: Mike X Cohen, sincxpress.com

COURSE URL: udemy.com/course/pycalc_x/?couponCode=202108

Start coding or [generate](#) with AI.

```
import numpy as np
import sympy as sym
import matplotlib.pyplot as plt

# better image resolution
from IPython import display
display.set_matplotlib_formats('svg')
```

```
/tmp/ipykernel_18955/3557205723.py:7: DeprecationWarning: `set_matplotlib_formats` is deprecated since IPython 7.23, directl
display.set_matplotlib_formats('svg')
```

Start coding or [generate](#) with AI.

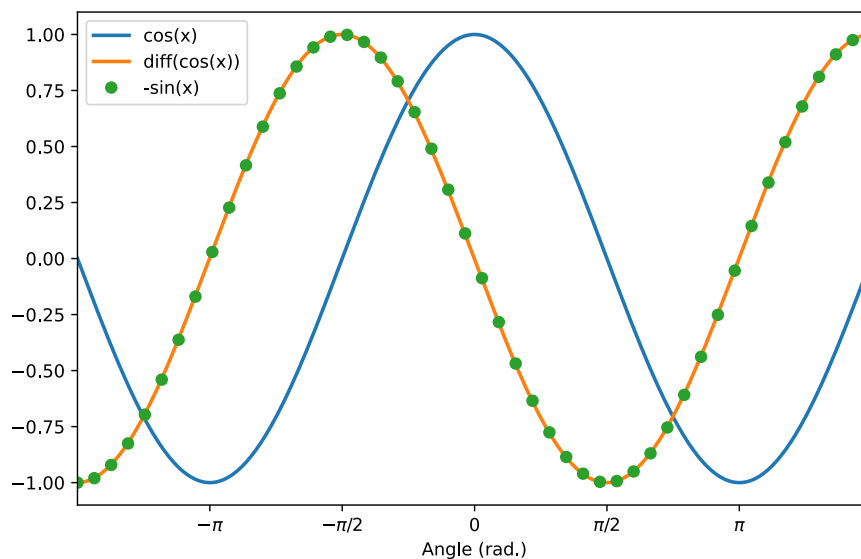
✓ Exercise 1: $d/dx \cos(x)$

```
dx = .01
x = np.arange(-1.5*np.pi,1.5*np.pi,step=dx)

# define the function and its difference (same as derivative?)
fx = np.cos(x)
dfx = np.diff(fx) / dx

plt.figure(figsize=(8,5))
plt.plot(x,fx,label='cos(x)',linewidth=2)
plt.plot(x[:-1],dfx,label='diff(cos(x))',linewidth=2)
plt.plot(x[::20],-np.sin(x[::20]),'o',label='-sin(x)')

plt.legend()
plt.xticks([-np.pi,-np.pi/2,0,np.pi/2,np.pi],labels=[r'$-\pi$',r'$-\pi/2$',r'$0$',r'$\pi/2$',r'$\pi$'])
plt.xlabel('Angle (rad.)')
plt.xlim(x[[0,-1]])
plt.show()
```



Start coding or [generate](#) with AI.

Exercise 2: $d/dx \sin(x)$

```

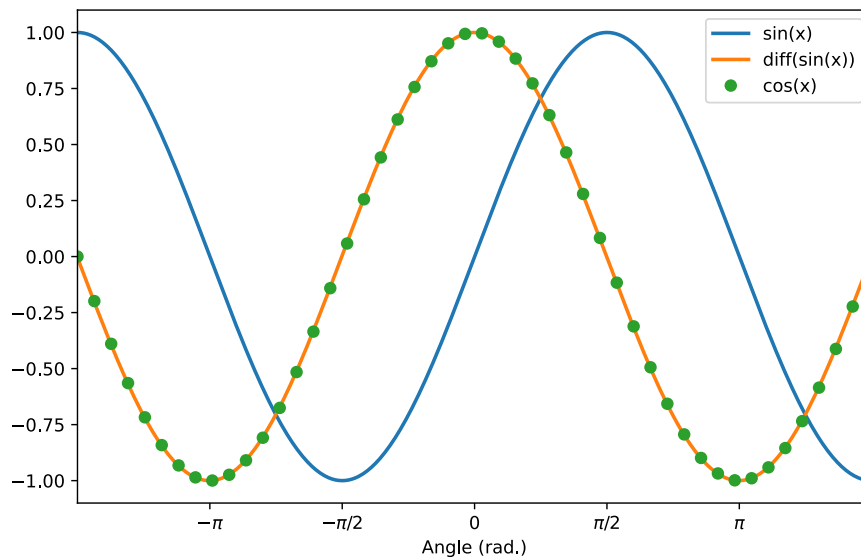
dx = .01
x = np.arange(-1.5*np.pi,1.5*np.pi,step=dx)

# define the function and its difference (same as derivative?)
fx = np.sin(x)
dfx = np.diff(fx) / dx

plt.figure(figsize=(8,5))
plt.plot(x,fx,label='sin(x)',linewidth=2)
plt.plot(x[:-1],dfx,label='diff(sin(x))',linewidth=2)
plt.plot(x[::20],np.cos(x[::20]),'o',label='cos(x)')

plt.legend()
plt.xticks([-np.pi,-np.pi/2,0,np.pi/2,np.pi],labels=[r'$-\pi$',r'$-\pi/2$',r'$0$',r'$\pi/2$',r'$\pi$'])
plt.xlabel('Angle (rad.)')
plt.xlim(x[[0,-1]])
plt.show()

```



Start coding or [generate](#) with AI.

Exercise 3: Cyclicity of trig derivatives

```

# symbolic variable
x = sym.symbols('x')

# start with cos
f = sym.cos(x)

# cycle through
for i in range(4):
    print(f"({f})' = {sym.diff(f)}")
    f = sym.diff(f)

(cos(x))' = -sin(x)
(-sin(x))' = -cos(x)
(-cos(x))' = sin(x)
(sin(x))' = cos(x)

```

Start coding or [generate](#) with AI.

Exercise 4: cyclic inverse trig derivatives

```

# start with acos
f = sym.acos(x)

# cycle through
for i in range(4):

```

```
print(f"({f})' = {sym.diff(f)}")
f = sym.diff(f)
```

```
(acos(x))' = -1/sqrt(1 - x**2)
(-1/sqrt(1 - x**2))' = -x/(1 - x**2)**(3/2)
(-x/(1 - x**2)**(3/2))' = -3*x**2/(1 - x**2)**(5/2) - 1/(1 - x**2)**(3/2)
(-3*x**2/(1 - x**2)**(5/2) - 1/(1 - x**2)**(3/2))' = -15*x**3/(1 - x**2)**(7/2) - 9*x/(1 - x**2)**(5/2)
```

Start coding or [generate](#) with AI.

Tệp tài liệu bạn tải lên là một bài giảng lập trình Python tập trung vào việc tìm **Điểm tới hạn (Critical points)** của hàm số bằng cả phương pháp số học (Numpy) và phương pháp ký hiệu (SymPy).

+3

Dưới đây là tóm tắt các nội dung chính của bài giảng:

1. Khái niệm Điểm tới hạn

Điểm tới hạn là những điểm trên đồ thị hàm số mà tại đó đạo hàm bằng 0 ($f'(x) = 0$) hoặc đạo hàm không tồn tại. Đây thường là các điểm cực đại hoặc cực tiểu của hàm số.

+1

2. Các phương pháp thực hành trong tệp

Phương pháp 1: Sử dụng Numpy và Scipy (Thực nghiệm)

Trong bài tập 1, tác giả sử dụng một mảng số để tìm điểm tới hạn của hàm $f(x) = x^2 + x^3$.

+1

- **Tính đạo hàm:** Sử dụng `np.diff(fx) / np.mean(np.diff(x))` để tính đạo hàm thực nghiệm.
- **Tìm điểm tới hạn:** Do phương pháp số ít khi cho kết quả đạo hàm bằng chính xác số 0, tác giả sử dụng thư viện `scipy.signal` với hàm `find_peaks` để tìm các cực trị (minima/maxima) của giá trị tuyệt đối đạo hàm.

+2

- **Kết quả:** Tìm được hai điểm tới hạn tại $x \approx -0.667$ và $x \approx 0.000$.

Phương pháp 2: Sử dụng SymPy (Ký hiệu toán học)

Bài tập 2 sử dụng thư viện `sympy` để giải bài toán một cách chính xác tuyệt đối.

+2

- **Hàm số:** $3x^2 + 2x$ (là đạo hàm của $x^2 + x^3$).
- **Giải phương trình:** Sử dụng `sym.solve(sym.Eq(df, 0))` để tìm nghiệm chính xác.
- **Kết quả chính xác:** Hai điểm tới hạn là $(-2/3, 4/27)$ và $(0, 0)$.

+1

Phương pháp 3: Điểm tới hạn với hàm không liên tục (Discontinuity)

Bài tập 3 xử lý một trường hợp phức tạp hơn là hàm phân đoạn (Piecewise).

+3

- **Hàm phân đoạn:**

- x^2 khi $x < 1$.

+1

- $-x^2/2$ khi $x \geq 1$.

+1

- **Xử lý:** Ngoài việc tìm nơi đạo hàm bằng 0, tác giả còn kiểm tra giới hạn tại điểm nối (breakpoint) bằng `sym.limit`. Nếu giới hạn trái và phải khác nhau (hàm không liên tục), điểm đó cũng có thể được coi là một điểm tới hạn đặc biệt.

+2

3. Các thư viện quan trọng được sử dụng:

+1

- **Numpy:** Xử lý mảng số và tính toán sai phân.
- **Sympy:** Giải phương trình đạo hàm chính xác dưới dạng phân số/ký hiệu.
- **Matplotlib:** Vẽ đồ thị hàm số và đạo hàm để trực quan hóa vị trí các điểm tới hạn.
- **Scipy:** Hỗ trợ tìm kiếm các đỉnh (peaks) trong dữ liệu số.

Tóm lại, bài giảng này hướng dẫn cách kết hợp giữa tư duy toán học thuần túy (Sympy) và kỹ năng xử lý dữ liệu thực tế (Numpy/Scipy) để xác định các điểm quan trọng trên một đường cong.

+2

✓ COURSE: Master calculus 1 using Python: derivatives and applications

SECTION: Differentiation fundamentals

LECTURE: CodeChallenge: Critical points in Python

TEACHER: Mike X Cohen, sincxpress.com

COURSE URL: udemy.com/course/pycalc1_x/?couponCode=202307

Start coding or [generate](#) with AI.

```
import numpy as np
import sympy as sym
import matplotlib.pyplot as plt

# better image resolution
import matplotlib_inline.backend_inline
matplotlib_inline.backend_inline.set_matplotlib_formats('svg')
```

Start coding or [generate](#) with AI.

✓ Exercise 1: Critical points in numpy

```
# x-axis grid
x = np.linspace(-1.5,1,10001)

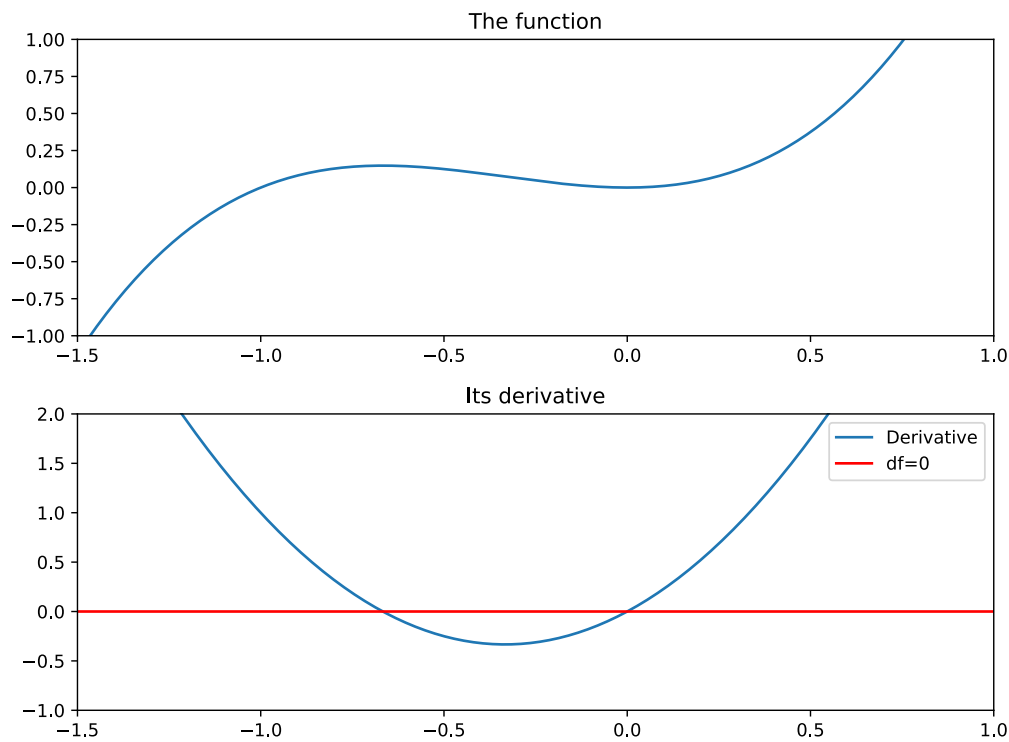
# function
fx = x**2 + x**3

# empirical deritivate (difference)
df = np.diff(fx) / np.mean(np.diff(x))

# plot
_,axs = plt.subplots(2,1,figsize=(8,6))
axs[0].plot(x,fx)
axs[0].set_ylim([-1,1])
axs[0].set_xlim(x[[0,-1]])
axs[0].set_title('The function')

axs[1].plot(x[:-1],df,label='Derivative')
axs[1].axhline(0,color='r',label='df=0')
axs[1].set_ylim([-1,2])
axs[1].set_xlim(x[[0,-1]])
axs[1].legend()
axs[1].set_title('Its derivative')

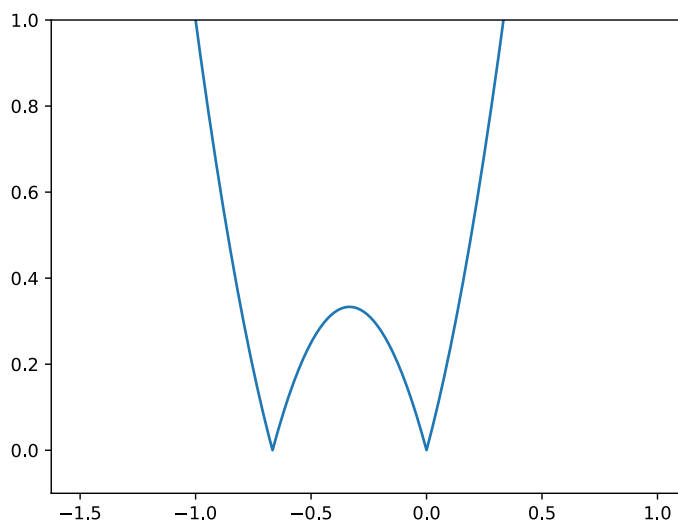
plt.tight_layout()
plt.show()
```



```
# find df=0
exactZero = np.where( df==0 )[0]
print(f'Critical points at x={exactZero}')
```

Critical points at x=[]

```
# setting up to find the minima
plt.plot(x[:-1], np.abs(df))
plt.ylim([-0.1, 1])
plt.show()
```



```
# find minima
from scipy.signal import find_peaks

# find the minima
dfLocalMin = find_peaks(-np.abs(df))[0]

# print the results
for xx,yy in zip(x[dfLocalMin], fx[dfLocalMin]):
    print(f'Critical point/value: ({xx:.3f}, {yy:.3f})')
```

Critical point/value: (-0.667, 0.148)
Critical point/value: (-0.000, 0.000)

```
# another plot
_,axs = plt.subplots(2,1,figsize=(8,6))
axs[0].plot(x,fx,label='Function')
```



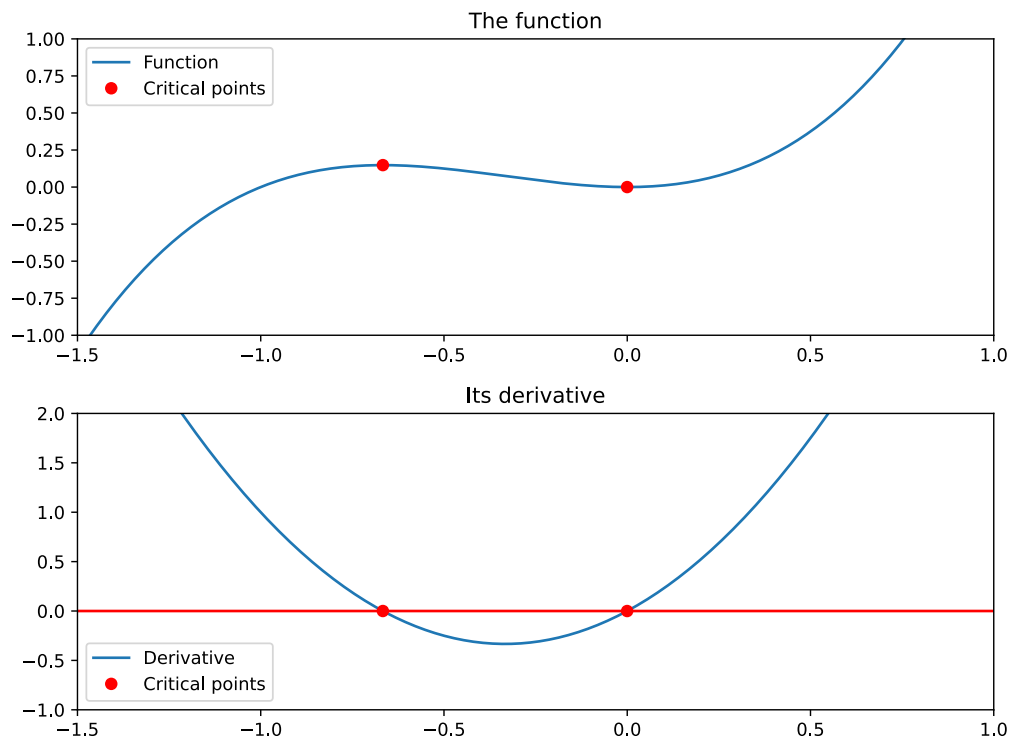
```

axs[0].plot(x[dfLocalMin],fx[dfLocalMin],'ro',label='Critical points')
axs[0].set_ylim([-1,1])
axs[0].set_xlim(x[[0,-1]])
axs[0].legend()
axs[0].set_title('The function')

axs[1].plot(x[:-1],df,label='Derivative')
axs[1].plot(x[dfLocalMin],df[dfLocalMin],'ro',label='Critical points')
axs[1].axhline(0,color='r')
axs[1].set_ylim([-1,2])
axs[1].set_xlim(x[[0,-1]])
axs[1].legend()
axs[1].set_title('Its derivative')

plt.tight_layout()
plt.show()

```



Start coding or [generate](#) with AI.

Exercise 2: Critical points in sympy

```

x = sym.symbols('x')

# the function and its derivative
fx = x**2 + x**3
df = sym.diff(fx)
df

```

$$3x^2 + 2x$$

```

# find and print the critical points
criticalPoints = sym.solve( sym.Eq(df,0) )

# print the results
for xx in criticalPoints:
    print(f'Critical point/value: ({xx}, {fx.subs(x,xx)})')

```

```

Critical point/value: (-2/3, 4/27)
Critical point/value: (0, 0)

```

Start coding or [generate](#) with AI.

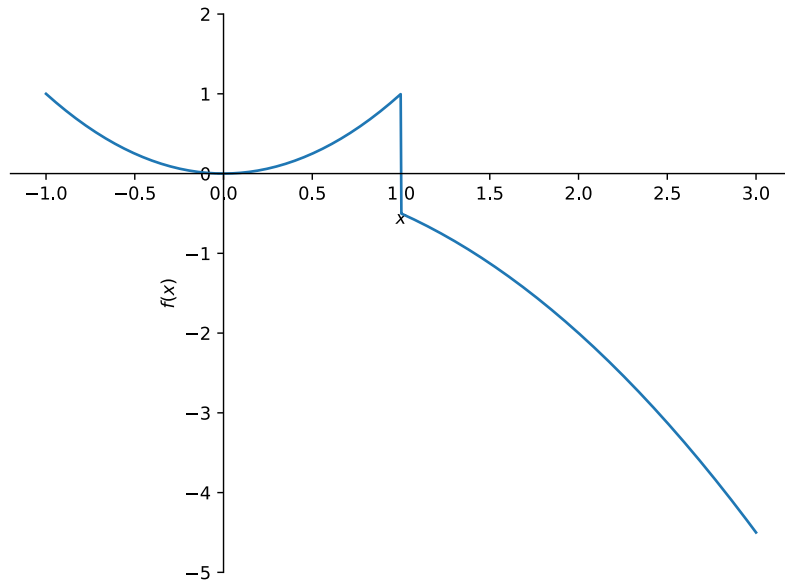
Exercise 3: Critical points with discontinuity

```
# list function pieces
piece1 = x**2
piece2 = -x**2/2

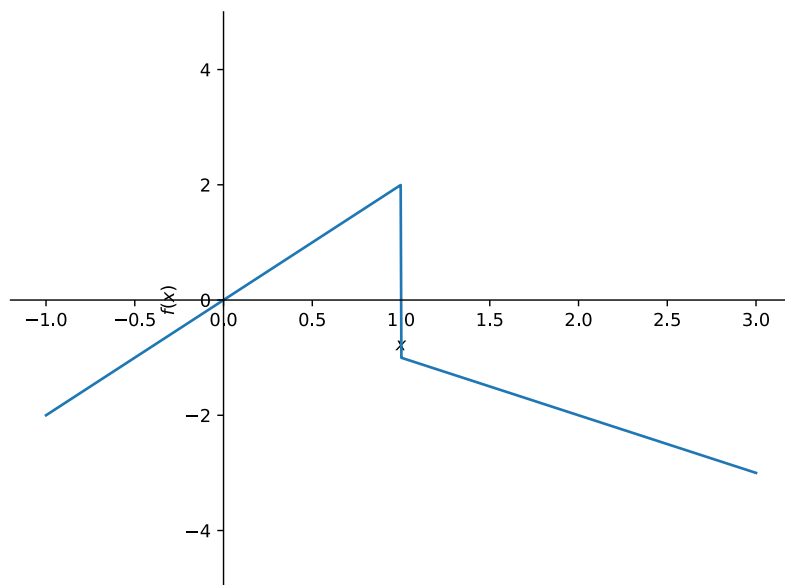
# boundary
bound = 1

# put them together with conditions
fx = sym.Piecewise(
    (piece1,x<bound),
    (piece2,x>=bound) )

sym.plot(fx,(x,-1,3),ylim=[-5,2]);
```



```
# compute and plot derivative
dfx = sym.diff(fx)
sym.plot(dfx,(x,-1,3),ylim=[-5,5]);
```



```
# find and print the critical points
criticalPoints = sym.solve( sym.Eq(dfx,0) )

criticalPoints
```

```
[0]
```

```
# find the limit at the breakpoints
limL = sym.limit(fx,x,bound,'-')
limR = sym.limit(fx,x,bound,'+')

if limL!=limR:
    criticalPoints.append( 1 )
```

```
print(f'Critical points: {criticalPoints}')
```

```
Critical points: [0]
```

Tập tài liệu bạn vừa tải lên là nội dung bài giảng về **Độ dốc Toàn cục (Global Slope)** và **Độ dốc Cục bộ (Local Slope)** trong lập trình Python. Đây là một phần quan trọng để hiểu cách đạo hàm hoạt động trên các đoạn nhỏ của một hàm số.

+1

Dưới đây là giải thích chi tiết các bài tập có trong file:

1. Bài tập 1: Độ dốc toàn cục (Global Slope)

Bài tập này hướng dẫn cách tính độ dốc của một đường thẳng đi qua hai điểm cố định.

- **Hai điểm:** $p1 = [-1, 1]$ và $p2 = [3, 6]$.
- **Công thức:** Độ dốc m được tính bằng sự chênh lệch trục tung chia cho sự chênh lệch trục hoành:

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

- **Kết quả:** Với dữ liệu trên, độ dốc tính được là **1.25**. Đồ thị hiển thị một đường thẳng nối liền hai điểm đỏ và xanh trên hệ trục tọa độ.

+1

2. Bài tập 2: Độ dốc cục bộ (Local Slopes)

Bài tập này mở rộng khái niệm độ dốc cho một hàm số (trong trường hợp này là hàm parabol $y = x^2$) bằng cách chia nó thành nhiều đoạn nhỏ.

+1

- **Cách thực hiện:**
 - Sử dụng `np.linspace` để tạo ra $N = 5$ điểm dữ liệu trên trục x từ -1 đến 5.
 - Sử dụng vòng lặp `for` để tính độ dốc cho từng đoạn thẳng nhỏ nối giữa hai điểm liên tiếp (x_{i-1} và x_i).
- **Kết quả đồ thị:** Tập hiển thị hai biểu đồ:
 - Biểu đồ trên là hàm số $y = x^2$ với các điểm rời rạc nối lại.

+1

- Biểu đồ dưới là giá trị độ dốc của từng phân đoạn (Segment slopes), cho thấy độ dốc thay đổi tùy theo vị trí của x .

+1

3. Bài tập 3: So sánh Độ dốc toàn cục và Trung bình cộng độ dốc cục bộ

Đây là phần thú vị nhất của bài giảng, chứng minh một tính chất toán học quan trọng.

- **Độ dốc toàn cục (Global):** Tính dựa trên điểm đầu tiên và điểm cuối cùng của toàn bộ dãy số.

+1

- **Trung bình độ dốc cục bộ (Ave of locals):** Tính bằng cách lấy trung bình cộng của tất cả các độ dốc nhỏ đã tính ở bài tập 2.

+2

- **Kết luận:** Cả hai giá trị đều bằng **4**. Điều này cho thấy trung bình cộng của các độ dốc thành phần chính bằng độ dốc nối giữa điểm đầu và điểm cuối.

Tóm tắt các kỹ thuật Python được sử dụng:

- **Numpy:** Dùng để tính toán mảng và giá trị trung bình (`np.mean`).

+1

- **Matplotlib:** Dùng để vẽ các đường thẳng phân đoạn và tùy chỉnh giao diện đồ thị (như đặt tỉ lệ trục bằng nhau bằng `set_aspect('equal')`).

+1

✓ COURSE: Master calculus 1 using Python: derivatives and applications

SECTION: Differentiation fundamentals

LECTURE: CodeChallenge: Global and local slopes

TEACHER: Mike X Cohen, sincxpress.com

COURSE URL: udemy.com/course/pycalc1_x/?couponCode=202307

Start coding or [generate](#) with AI.

```
import numpy as np
import sympy as sym
import matplotlib.pyplot as plt

# better image resolution
import matplotlib_inline.backend_inline
matplotlib_inline.backend_inline.set_matplotlib_formats('svg')
```

Start coding or [generate](#) with AI.

✓ Exercise 1: global slope

```
# define points as XY pairs
p1 = [-1,1]
p2 = [3,6]

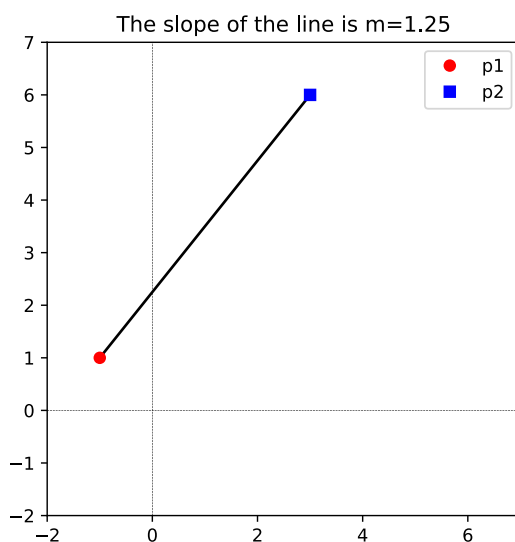
# compute slope
m = (p2[1]-p1[1]) / (p2[0]-p1[0])

# plot the dots and a line between them
plt.plot([p1[0],p2[0]],[p1[1],p2[1]], 'k')
plt.plot(p1[0],p1[1], 'ro', label='p1')
plt.plot(p2[0],p2[1], 'bs', label='p2')

# put the slope in the title
plt.title(f'The slope of the line is m={m}')

# make the plot look a bit nicer
plt.plot([-2,7],[0,0], 'k--', linewidth=.3)
plt.plot([0,0],[-2,7], 'k--', linewidth=.3)

plt.xlim([-2,7])
plt.ylim([-2,7])
plt.legend()
plt.gca().set_aspect('equal')
plt.show()
```



Start coding or [generate](#) with AI.

Exercise 2: local slopes

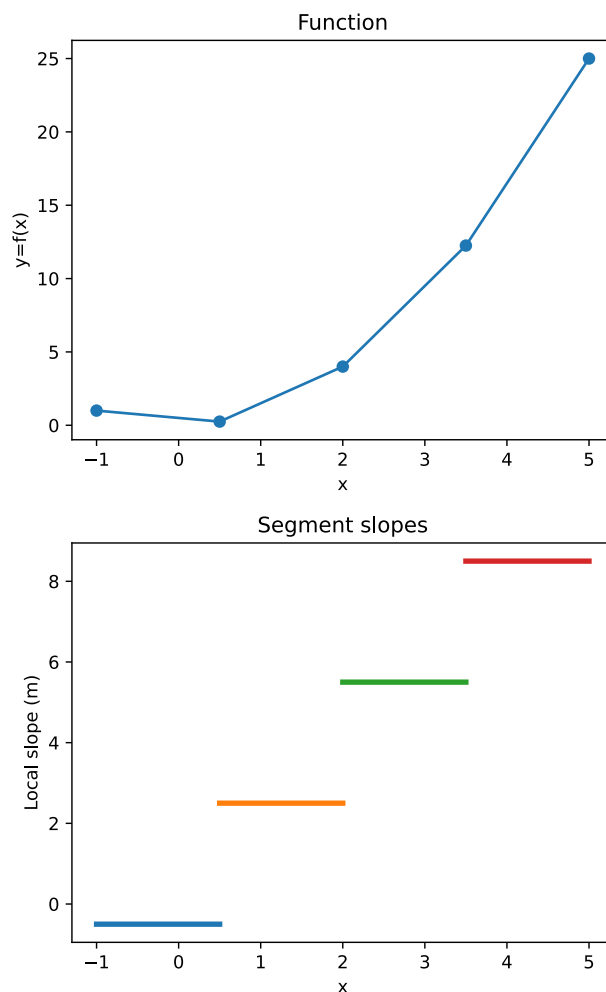
```
# define function
N = 5
x = np.linspace(-1,5,N)
y = x**2

_,axs = plt.subplots(2,1,figsize=(5,8))
axs[0].plot(x,y,'o-')
axs[0].set_xlabel('x')
axs[0].set_ylabel('y=f(x)')
axs[0].set_title('Function')

# compute and plot the slopes
m = np.zeros(N-1)
for i in range(1,N):
    m[i-1] = y[i]-y[i-1]
    m[i-1] /= x[i]-x[i-1]
    axs[1].plot([x[i-1],x[i]],m[i-1],m[i-1]],linewidth=3)

axs[1].set_title('Segment slopes')
axs[1].set_xlabel('x')
axs[1].set_ylabel('Local slope (m)')

plt.tight_layout()
plt.show()
```



Start coding or [generate](#) with AI.

Exercise 3: Global and average of local slopes

```
globalSlope = (y[-1]-y[0]) / (x[-1]-x[0])  
aveLocalSlopes = np.mean(m)
```

```
print('      Global: %g' %globalSlope)  
print('Ave of locals: %g' %aveLocalSlopes)
```

```
      Global: 4  
Ave of locals: 4
```

Start coding or [generate](#) with AI.